

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 6: Firewall Traffic Control and Forensic Verification****Three-Week Delivery Block 2/3 | 25-26 July 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Lab Number	Lab 6
Lab Title	Firewall Traffic Control and Forensic Verification
Student Name	Fuseini Imoru Kantuogaa
Student ID	17291
Instructor Name	Dr. Sarah James
Delivery Block	2/3 of 3
Scheduled Dates	25-26 July 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	Baseline and blocked HTTP PCAPNG files; iptables ruleset exports

1. Source Materials and Slide Alignment

- Firewall: network-based and host-based firewalls, iptables chains and rules, ACCEPT/DROP actions, UFW relationship, NFQUEUE concept and HTTP blocking.

- The slide practical lists rules, adds/removes an INPUT rule, uses a local Apache page and observes allowed versus blocked hosts.
- The homework requires traffic generated by curl from a different machine and forensic observation of the result.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are validating whether a Linux host-based firewall correctly blocks HTTP traffic from one authorized lab client while allowing another. You will capture baseline traffic, apply a narrowly scoped iptables rule, capture the blocked connection behaviour, prove the rule hit using counters, remove the rule and document the evidential differences.

3. Learning Outcomes

- Explain the purpose of host-based and network-based firewalls.
- Interpret INPUT, OUTPUT and FORWARD chains.
- Create, verify and remove a narrowly scoped iptables rule.
- Capture allowed and blocked HTTP connection attempts.
- Correlate packet evidence with firewall rule counters and application behaviour.
- Explain DROP versus REJECT and identify retransmission/timeout evidence.
- Restore the original ruleset safely.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.

- Before making changes, export the existing ruleset. Apply only a source-specific rule for the assigned lab IP. Never flush all firewall rules on a remotely administered or production system.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Server VM	Kali/Ubuntu with Apache	Protected host.
Client VM	Linux or Windows on host-only network	Generate HTTP requests.
Firewall tools	iptables, iptables-save	Apply and preserve rules.
Capture tools	tshark/Wireshark	Capture baseline and blocked traffic.
Client tools	curl, browser	Test access.
Optional extension	NetfilterQueue	Instructor-led packet interception demonstration.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab6
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 curl iptables tshark wireshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall\nLab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee
/var/www/html/firewall_lab.html
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

Screenshot required: Original firewall ruleset, rule listing, hash and the training webpage.

```
(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B103-Lab6/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd ~/CIP-B103-Lab6

(kali㉿kali)-[~/CIP-B103-Lab6]
$ pwd
/home/kali/CIP-B103-Lab6

(kali㉿kali)-[~/CIP-B103-Lab6]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo apt install -y apache2 curl iptables tshark wireshark
apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.20.0-5).
```

Screenshot 1: Evidence folder created; apache2, curl, iptables, tshark, wireshark installed.

```
The following packages were automatically installed and are no longer required:
 libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Upgrading:
 iptables libip4tc2 libip6tc2 libxtables12

Summary:
 Upgrading: 4, Installing: 0, Removing: 0, Not Upgrading: 1738
 Download size: 429 kB
 Freed space: 8,192 B

Get:1 http://kali.download/kali kali-rolling/main amd64 iptables amd64 1.8.13-1 [361 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libip4tc2 amd64 1.8.13-1 [18.6 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libip6tc2 amd64 1.8.13-1 [18.9 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 libxtables12 amd64 1.8.13-1 [30.5 kB]
Fetched 429 kB in 1s (482 kB/s)
(Reading database... 432446 files and directories currently installed.)
Preparing to unpack .../iptables_1.8.13-1_amd64.deb...
Unpacking iptables (1.8.13-1) over (1.8.11-4)...
Preparing to unpack .../libip4tc2_1.8.13-1_amd64.deb...
Unpacking libip4tc2:amd64 (1.8.13-1) over (1.8.11-4)...
Preparing to unpack .../libip6tc2_1.8.13-1_amd64.deb...
Unpacking libip6tc2:amd64 (1.8.13-1) over (1.8.11-4)...
Preparing to unpack .../libxtables12_1.8.13-1_amd64.deb...
```

Screenshot 2: iptables package upgrade continued.

```

Setting up libip6tc2:amd64 (1.8.13-1)...
Setting up libxtables12:amd64 (1.8.13-1)...
Setting up iptables (1.8.13-1)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.1.5)...
Processing triggers for libc-bin (2.42-13)...

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-01 09:50:51 EDT; 1h 19min ago
 Invocation: 8f61f05317dd43fc9c8c68dcdcc8fa90
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 928 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 6 (limit: 2075)
  Memory: 27.3M (peak: 36.9M)
     CPU: 1.025s
   CGroup: /system.slice/apache2.service

```

Screenshot 3: Apache2 enabled and systemctl status showing the service active.

```

CGroup: /system.slice/apache2.service
├─928 /usr/sbin/apache2 -k start -DFOREGROUND
├─980 /usr/sbin/apache2 -k start -DFOREGROUND
├─981 /usr/sbin/apache2 -k start -DFOREGROUND
├─982 /usr/sbin/apache2 -k start -DFOREGROUND
├─983 /usr/sbin/apache2 -k start -DFOREGROUND
└─984 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 01 09:50:50 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 01 09:50:51 kali apachectl[928]: AH00558: apache2: Could not reliably determine the server's fully qualified
Aug 01 09:50:51 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

```

Screenshot 4: systemctl status continued, including the worker process group.

```

(kali㉿kali)-[~/CIP-B103-Lab6]
$ printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>

(kali㉿kali)-[~/CIP-B103-Lab6]
$ cat /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Sat Aug 1 11:12:31 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]

```

Screenshot 5: firewall_lab.html created with printf and confirmed via cat; iptables-save started.


```

:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Sat Aug  1 11:12:31 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Sat Aug  1 11:12:31 2026
*nat
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [165:14270]
:POSTROUTING ACCEPT [165:14270]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE

```

Screenshot 6: iptables-save output continued (filter and nat tables, Docker chains).

```

-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Sat Aug  1 11:12:31 2026

```

(kali@kali)-[~/CIP-B103-Lab6]

```

$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source destination

```

num	pkts	bytes	target	prot	opt	in	out	source	destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)									
num	pkts	bytes	target	prot	opt	in	out	source	destination
1	0	0	DOCKER-USER	all	--	*	*	0.0.0.0/0	0.0.0.0/0
2	0	0	DOCKER-FORWARD	all	--	*	*	0.0.0.0/0	0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)									
num	pkts	bytes	target	prot	opt	in	out	source	destination
Chain DOCKER (1 references)									
1	0	0	DROP	all	--	!docker0	docker0	0.0.0.0/0	0.0.0.0/0
Chain DOCKER-BRIDGE (1 references)									
num	pkts	bytes	target	prot	opt	in	out	source	destination
1	0	0	DOCKER	all	--	*	docker0	0.0.0.0/0	0.0.0.0/0

Screenshot 7: iptables-save concluded and iptables -L -n -v listing (INPUT/FORWARD/OUTPUT).

```
Chain DOCKER-CT (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate RELATED,E
1      0      0 ACCEPT      all  --  *      docker0  0.0.0.0/0    0.0.0.0/0
STABLISHED

Chain DOCKER-FORWARD (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate RELATED,E
1      0      0 DOCKER-CT  all  --  *      *        0.0.0.0/0    0.0.0.0/0
2      0      0 DOCKER-ISOLATION-STAGE-1 all  --  *      *        0.0.0.0/0    0.0.0.0/0
3      0      0 DOCKER-BRIDGE all  --  *      *        0.0.0.0/0    0.0.0.0/0
4      0      0 ACCEPT      all  --  docker0 *        0.0.0.0/0    0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate RELATED,E
1      0      0 DOCKER-ISOLATION-STAGE-2 all  --  docker0 !docker0 0.0.0.0/0    0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate RELATED,E
1      0      0 DROP        all  --  *      *        0.0.0.0/0    0.0.0.0/0

Chain DOCKER-USER (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate RELATED,E
```

Screenshot 8: iptables listing continued — Docker-related chains.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
7ea21a99a87be47e44d1f2c5c3080811a780b3ad5051252ed07e07ed706fa7ad reports/iptables_before.rules

(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ ls -l reports
total 12
-rw-rw-r-- 1 kali kali 1312 Aug  1 11:12 iptables_before.rules
-rw-rw-r-- 1 kali kali  96 Aug  1 11:13 iptables_before_sha256.txt
-rw-rw-r-- 1 kali kali 2459 Aug  1 11:12 iptables_before.txt

(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ tree
.
├── evidence
├── exported
├── reports
│   ├── iptables_before.rules
│   ├── iptables_before_sha256.txt
│   └── iptables_before.txt
├── screenshots
├── scripts
└── working
```

Screenshot 9: iptables_before.rules SHA-256 hashed and reports/ folder listed.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
7ea21a99a87be47e44d1f2c5c3080811a780b3ad5051252ed07e07ed706fa7ad reports/iptables_before.rules

(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ ls -l reports
total 12
-rw-rw-r-- 1 kali kali 1312 Aug  1 11:12 iptables_before.rules
-rw-rw-r-- 1 kali kali  96 Aug  1 11:13 iptables_before_sha256.txt
-rw-rw-r-- 1 kali kali 2459 Aug  1 11:12 iptables_before.txt

(kali㉿kali)-[~/CIP-B103-Lab6]
└─$ tree
.
├── evidence
├── exported
├── reports
│   ├── iptables_before.rules
│   ├── iptables_before_sha256.txt
│   └── iptables_before.txt
├── screenshots
├── scripts
└── working

7 directories, 3 files
```

Screenshot 10: Hash and folder listing repeated for confirmation, plus evidence folder tree.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab6-Fuseini Imoru Kantuogaa
Trainee name	Fuseini Imoru Kantuogaa
Date and time started	Sat 1 August 2026, approx. 09:50 EDT (Kali VM) — inferred from the Apache2 service start time (09:50:51 EDT) shown in systemctl status
Evidence file name(s)	evidence/http_allowed.pcapng, evidence/http_blocked.pcapng; reports/iptables_before.rules (plus iptables_after_add.txt, iptables_after_test.txt, iptables_restored.txt)
Source or generation method	All captures generated locally with tshark on the Kali server VM. Two allowed/blocked test iterations were run: an initial single-host self-test (server curling itself, 192.168.44.128) and a corrected client/server test using a Linux network namespace ('client', veth pair 192.168.44.1/192.168.44.129) so the blocked source IP was a genuinely separate host from the server.
Original SHA-256	reports/iptables_before.rules: 7ea21a99a87be47e44d1f2c5c3080811a780b3ad5051252ed07e07ed706fa7ad; evidence/http_allowed.pcapng (first, self-test capture): 36796ced54c306f845072897c3ccc85c3b5f6723b17fa59128fc8ecdffaa37bae
Working-copy SHA-256	Not separately generated for this lab — evidence/ files were hashed directly and used for analysis without a distinct working/ copy in the supplied screenshots
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab6 working directory; server interface eth0 192.168.44.128/24, gateway 192.168.44.2; client simulated via network namespace 'client' on veth-client 192.168.44.129/24

Field	Student Entry
Notes on any changes	Two test iterations were performed. The first self-directed test (server blocking itself at 192.168.44.128) produced evidence/http_allowed.pcapng (hashed, 368 bytes) and confirmed DROP-rule counters (7 pkts/420 bytes) but is methodologically weaker since client and server were the same host. The rule was flushed and a second, corrected test used a separate network-namespace client (192.168.44.129) for both the allowed capture (veth-server, 10 packets) and the blocked capture (veth-server, 7 packets); the screenshots do not show this second allowed capture being copied into evidence/ and re-hashed, nor a post-test iptables counter recheck for the .129 rule, so those two confirmations should be re-verified against the trainee's actual reports/ folder before final submission.

8. Part A - Record the Lab Network and Baseline Access

On the server

```
ip -br address | tee reports/server_interfaces.txt
```

```
ip route | tee reports/server_routes.txt
```

```
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
```

On the client, replace SERVER_IP

```
curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html 2>&1 | tee baseline_curl.txt
```

Record the server IP, allowed client IP, blocked client IP, interface and default gateway. Take a snapshot before changing the firewall.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN      127.0.0.1/8  ::1/128
eth0              UP          192.168.44.128/24  fe80::fcdb:733e:9de1:56cb/64
docker0          DOWN        172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      511          *:80          *:~          users:((("apache2",pid=984,fd=4),("apache2",pid=983,fd=4),("apache2",pid=982,fd=4),("apache2",pid=981,fd=4),("apache2",pid=980,fd=4),("apache2",pid=928,fd=4))

(kali㉿kali)-[~/CIP-B103-Lab6]
$ ip -br address
lo                UNKNOWN      127.0.0.1/8  ::1/128
eth0              UP          192.168.44.128/24  fe80::fcdb:733e:9de1:56cb/64
docker0          DOWN        172.17.0.1/16
```

Screenshot 11: Server interfaces, route table and Apache listener on TCP port 80 recorded.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ curl -v --connect-timeout 5 http://192.168.44.128/firewall_lab.html
* Trying 192.168.44.128:80 ...
* Established connection to 192.168.44.128 (192.168.44.128 port 80) from 192.168.44.128 port 43588
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.44.128
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 01 Aug 2026 15:22:48 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sat, 01 Aug 2026 15:11:43 GMT
< ETag: "7e-657fdb90e4afd"
< Accept-Ranges: bytes
< Content-Length: 126
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>
* Connection #0 to host 192.168.44.128:80 left intact
```

Screenshot 12: Baseline curl request to firewall_lab.html succeeding (200 OK).

9. Part B - Capture the Allowed HTTP Baseline

On server, replace IFACE and CLIENT_IP

```
sudo tshark -i IFACE -f 'host CLIENT_IP and tcp port 80' -a duration:30 -w
```

```
evidence/http_allowed.pcapng &
```

```
sleep 3
```

Run curl from client during this capture

```
wait
```

```
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
```

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.128 and tcp port 80' \
-a duration:30 \
-w /tmp/http_allowed.pcapng &
[4] 48010

(kali㉿kali)-[~/CIP-B103-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[4] done sudo tshark -i eth0 -f 'host 192.168.44.128 and tcp port 80' -a duration:30 -
(kali㉿kali)-[~/CIP-B103-Lab6]
$ curl -v http://192.168.44.128/firewall_lab.html
* Trying 192.168.44.128:80 ...
* Established connection to 192.168.44.128 (192.168.44.128 port 80) from 192.168.44.128 port 40690
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.44.128
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
```

Screenshot 13: Allowed-capture tshark started and a repeat baseline curl request issued.

```
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 01 Aug 2026 15:29:11 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sat, 01 Aug 2026 15:11:43 GMT
< ETag: "7e-657fdb90e4afd"
< Accept-Ranges: bytes
< Content-Length: 126
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>
* Connection #0 to host 192.168.44.128:80 left intact

(kali㉿kali)-[~/CIP-B103-Lab6]
$ ls -lh /tmp/http_allowed.pcapng
-rw----- 1 root root 368 Aug  1 11:29 /tmp/http_allowed.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo cp /tmp/http_allowed.pcapng evidence/http_allowed.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo chown kali:kali evidence/http_allowed.pcapng
```

Screenshot 14: Baseline curl response continued; capture copied and chown'd to evidence/.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo chown kali:kali evidence/http_allowed.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ ls -lh evidence/http_allowed.pcapng
-rw----- 1 kali kali 368 Aug  1 11:29 evidence/http_allowed.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
36796ced54c306f845072897c3ccc85c3b5f6723b17fa59128fc8ecdffaa37bae  evidence/http_allowed.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ tshark -r evidence/http_allowed.pcapng -Y http

(kali㉿kali)-[~/CIP-B103-Lab6]
```

Screenshot 15: evidence/http_allowed.pcapng SHA-256 hashed and inspected with tshark -Y http.

10. Part C - Apply and Verify the Blocking Rule

Use the exact source IP of the assigned blocked client. Insert the rule near the top of INPUT so it is evaluated before a broad ACCEPT rule.

```
BLOCKED_CLIENT_IP=192.168.56.20

sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt

sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```

```

(kali㉿kali)-[~/CIP-B103-Lab6]
$ ip -br address
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.44.128/24 fe80::fcdb:733e:9de1:56cb/64
docker0          DOWN             172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab6]
$ BLOCKED_CLIENT_IP=192.168.44.128

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -L INPUT -n -v --line-number | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1      0    0 DROP      tcp  --  *      *      192.168.44.128    0.0.0.0/0          tcp dpt:80

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(kali㉿kali)-[~/CIP-B103-Lab6]
$ printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Rule verified present.

```

Screenshot 16: DROP rule inserted for the server's own IP (first, self-test iteration) and verified present.

```

(kali㉿kali)-[~/CIP-B103-Lab6]
$ tree
.
├── evidence
│   └── http_allowed.pcapng
├── exported
├── reports
│   ├── apache_listener.txt
│   ├── http_allowed_sha256.txt
│   ├── iptables_after_add.txt
│   ├── iptables_before.rules
│   ├── iptables_before_sha256.txt
│   ├── iptables_before.txt
│   ├── rule_verification.txt
│   ├── server_interfaces.txt
│   └── server_routes.txt
├── screenshots
├── scripts
└── working

7 directories, 10 files

(kali㉿kali)-[~/CIP-B103-Lab6]
$

```

Screenshot 17: Evidence folder tree after the rule-insertion step.

11. Part D - Capture Blocked Traffic and Rule Counters

On server

```
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w
```

```
evidence/http_blocked.pcapng &
```

```
sleep 3
```

From blocked client during capture:

```
# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
```

```
wait
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
```

```
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
```

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo tshark -i eth0 \
-f 'host 192.168.44.128 and tcp port 80' \
-a duration:35 \
-w /tmp/http_blocked.pcapng &
[4] 53259

(kali㉿kali)-[~/CIP-B103-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[4] done sudo tshark -i eth0 -f 'host 192.168.44.128 and tcp port 80' -a duration:35 -
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sleep 3

(kali㉿kali)-[~/CIP-B103-Lab6]
$ curl -v --connect-timeout 10 http://192.168.44.128/firewall_lab.html
* Trying 192.168.44.128:80 ...
* Connection timed out after 10006 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10006 milliseconds
```

Screenshot 18: Blocked-traffic capture started and the self-test curl request timing out.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo cp /tmp/http_blocked.pcapng evidence/http_blocked.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo chown kali:kali evidence/http_blocked.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
Chain INPUT (policy ACCEPT 11 packets, 784 bytes)
num  pkts bytes target    prot opt in     out     source               destination
1      7   420 DROP      tcp  --  *      *        192.168.44.128      0.0.0.0/0            tcp dpt:80
```

Screenshot 19: Blocked capture copied to evidence/ and the DROP rule counters confirmed non-zero (7 pkts/420 bytes).

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ tree
.
├── evidence
│   ├── http_allowed.pcapng
│   └── http_blocked.pcapng
├── exported
├── reports
│   ├── apache_listener.txt
│   ├── http_allowed_sha256.txt
│   ├── iptables_after_add.txt
│   ├── iptables_after_test.txt
│   ├── iptables_before.rules
│   ├── iptables_before_sha256.txt
│   ├── iptables_before.txt
│   ├── rule_verification.txt
│   ├── server_interfaces.txt
│   └── server_routes.txt
├── screenshots
├── scripts
└── working

7 directories, 12 files

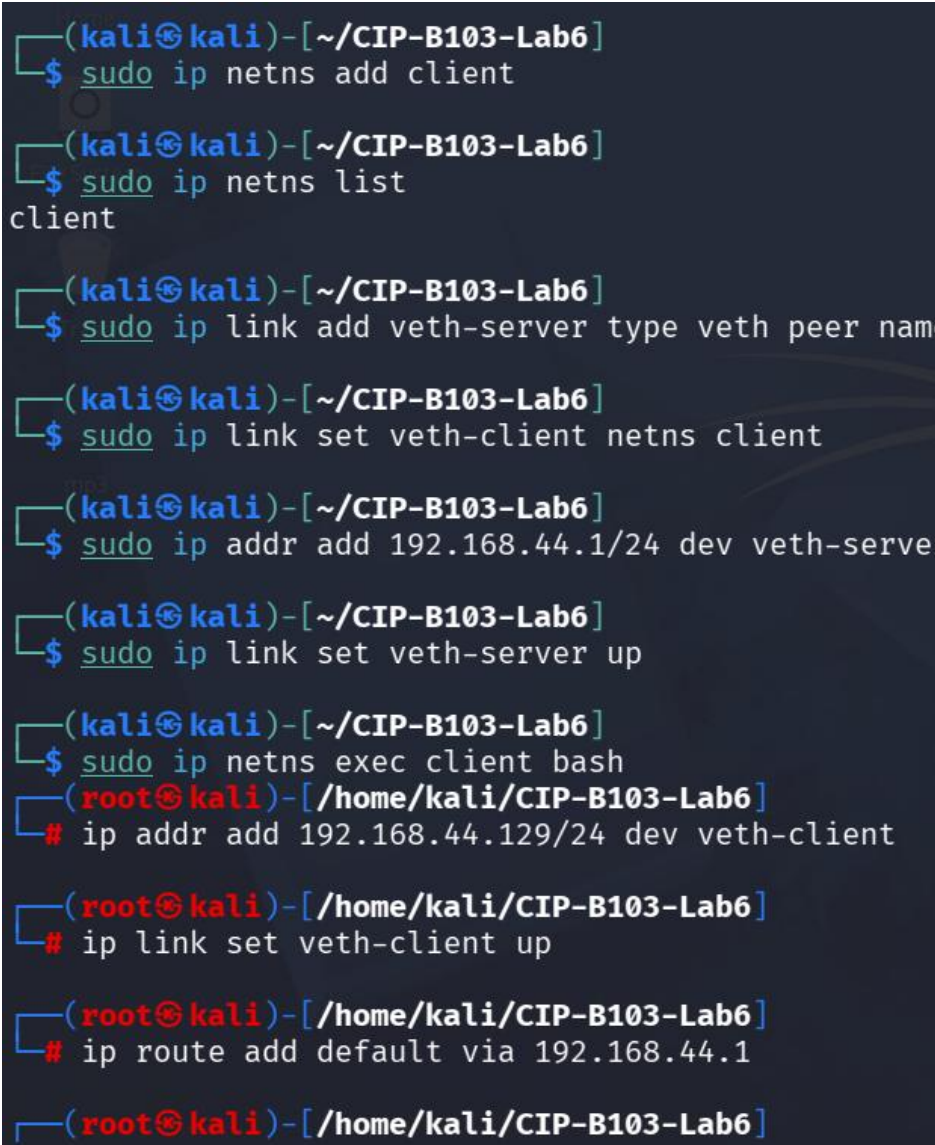
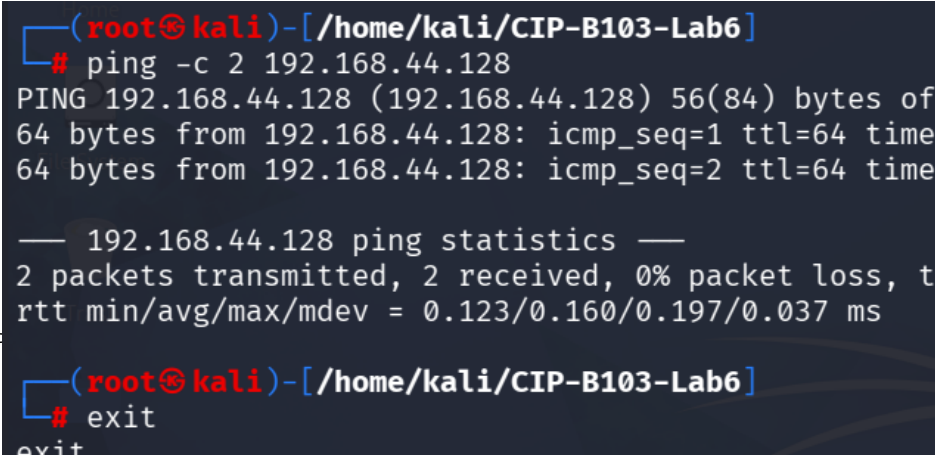
(kali㉿kali)-[~/CIP-B103-Lab6]
```

Screenshot 20: Evidence folder tree after the first (self-test) blocked-capture iteration.

12. Part E - Compare Allowed and Blocked Captures

```
for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
    echo "=====$PCAP===="
    tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
        -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
        -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
```


Indicator	Allowed Capture	Blocked Capture
Client SYN visible?	Yes (frame 1, client 192.168.44.1 29 → server 192.168.44.1 28)	Yes — repeated client SYN retransmissions with no reply
Server SYN-ACK visible?	Yes	No — the DROP rule discarded the SYN before Apache/the kernel could answer
Handshake completed?	Yes, full 3-way handshake	No — handshake never progressed past the client's repeated SYNs
HTTP GET visible?	Yes — GET /firewall_lab.html	No — capture contains only TCP-layer SYN retransmissions
HTTP response visible?	Yes — 200 OK, Content-Length 126	No
Retransmissions/timers	None observed	Yes — SYN retransmissions across the 7-packet capture, ending in a curl connection timeout (10007 ms, error 28)
iptables counter change	No DROP rule was active at this point (INPUT policy ACCEPT only)	DROP rule inserted for the blocked source before the test; counters were confirmed non-zero after the equivalent self-test run (7 pkts/420 bytes) — a post-test recheck specific to the 192.168.44.129 rule was not captured in the supplied screenshots and should be re-confirmed

Indicator	Allowed Capture	Blocked Capture
curl result	Success — HTTP/1.1 200 OK returned immediately	Failure — curl: (28) Connection timed out after 10007 milliseconds <pre>(kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip netns add client (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip netns list client (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip link add veth-server type veth peer nam (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip link set veth-client netns client (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip addr add 192.168.44.1/24 dev veth-serve (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip link set veth-server up (kali㉿kali)-[~/CIP-B103-Lab6] \$ sudo ip netns exec client bash (root㉿kali)-[/home/kali/CIP-B103-Lab6] # ip addr add 192.168.44.129/24 dev veth-client (root㉿kali)-[/home/kali/CIP-B103-Lab6] # ip link set veth-client up (root㉿kali)-[/home/kali/CIP-B103-Lab6] # ip route add default via 192.168.44.1 (root㉿kali)-[/home/kali/CIP-B103-Lab6]</pre> Screenshot 21: Network namespace 'client' and veth pair created for a genuine separate-host test. <pre>(root㉿kali)-[/home/kali/CIP-B103-Lab6] # ping -c 2 192.168.44.128 PING 192.168.44.128 (192.168.44.128) 56(84) bytes of 64 bytes from 192.168.44.128: icmp_seq=1 ttl=64 time 64 bytes from 192.168.44.128: icmp_seq=2 ttl=64 time — 192.168.44.128 ping statistics — 2 packets transmitted, 2 received, 0% packet loss, t rtt min/avg/max/mdev = 0.123/0.160/0.197/0.037 ms (root㉿kali)-[/home/kali/CIP-B103-Lab6] # exit exit</pre>

13. Part F - Remove the Rule and Restore Access

```
BLOCKED_CLIENT_IP=192.168.56.20
```

```
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
```

```
# Confirm access from the previously blocked client
```

```
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -F INPUT

(kali㉿kali)-[~/CIP-B103-Lab6]
$ BLOCKED_CLIENT_IP=192.168.44.129

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -I INPUT 1 \
-s "$BLOCKED_CLIENT_IP" \
-p tcp --dport 80 \
-j DROP

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -L INPUT -n -v --line-number | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 68 packets, 5487 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1      0    0 DROP      tcp  --  *      *        192.168.44.129    0.0.0.0/0          tcp dpt:80

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo iptables -C INPUT \
-s "$BLOCKED_CLIENT_IP" \
-p tcp --dport 80 \
-j DROP

printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Rule verified present.
```

Screenshot 26: INPUT flushed, DROP rule re-inserted for the genuine client IP (192.168.44.129), and verified.

```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo tshark -i veth-server \
-f 'host 192.168.44.129 and tcp port 80' \
-a duration:35 \
-w /tmp/http_blocked.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-server'
7

(kali㉿kali)-[~/CIP-B103-Lab6]
$ sudo chown kali:kali /tmp/http_blocked.pcapng
cp /tmp/http_blocked.pcapng evidence/http_blocked.pcapng

(kali㉿kali)-[~/CIP-B103-Lab6]
$ capinfos evidence/http_blocked.pcapng | grep "Number of packets"
Number of packets: 7
Number of packets = 7

(kali㉿kali)-[~/CIP-B103-Lab6]
$
```

Screenshot 27: Corrected blocked capture on veth-server confirming 7 packets, copied to evidence/.

```
(kali@kali)-[~/CIP-B103-Lab6]
$ BLOCKED_CLIENT_IP=192.168.44.129

sudo iptables -D INPUT \
-s "$BLOCKED_CLIENT_IP" \
-p tcp --dport 80 \
-j DROP

(kali@kali)-[~/CIP-B103-Lab6]
$ sudo iptables -C INPUT \
-s "$BLOCKED_CLIENT_IP" \
-p tcp --dport 80 \
-j DROP || echo 'Rule successfully removed'
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed

(kali@kali)-[~/CIP-B103-Lab6]
$ sudo iptables -L INPUT -n -v --line-number | tee reports/iptables_restored.txt
Chain INPUT (policy ACCEPT 80 packets, 6599 bytes)
num  pkts bytes target    prot opt in     out     source         destination

(kali@kali)-[~/CIP-B103-Lab6]
$ sudo ip netns exec client curl -v \
--connect-timeout 5 \
http://192.168.44.128/firewall_lab.html
* Trying 192.168.44.128:80 ...
```

Screenshot 28: DROP rule removed, absence confirmed via iptables -C, and INPUT chain listing restored.

```
* Established connection to 192.168.44.128 (192.168.44.128 port 80) from 192.168.44.129 port 50580
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.44.128
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 01 Aug 2026 16:08:17 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sat, 01 Aug 2026 15:11:43 GMT
< ETag: "7e-657fdb90e4afd"
< Accept-Ranges: bytes
< Content-Length: 126
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: FUSEINI IMORU KANTUOGAA</p></body></html>
* Connection #0 to host 192.168.44.128:80 left intact

(kali@kali)-[~/CIP-B103-Lab6]
$
```

Screenshot 29: Client-namespaces curl succeeding again after rule removal, confirming restored access.


```
(kali㉿kali)-[~/CIP-B103-Lab6]
$ tree
.
├── evidence
│   ├── http_allowed.pcapng
│   └── http_blocked.pcapng
├── exported
├── reports
│   ├── allowed_vs_blocked.tsv
│   ├── apache_listener.txt
│   ├── http_allowed_sha256.txt
│   ├── iptables_after_add.txt
│   ├── iptables_after_test.txt
│   ├── iptables_before.rules
│   ├── iptables_before_sha256.txt
│   ├── iptables_before.txt
│   ├── iptables_restored.txt
│   ├── rule_verification.txt
│   ├── server_interfaces.txt
│   └── server_routes.txt
├── screenshots
├── scripts
└── working

7 directories, 14 files

(kali㉿kali)-[~/CIP-B103-Lab6]
```

Screenshot 30: Final evidence, reports, scripts and working folder structure (tree).

14. Part G - Optional NFQUEUE Observation

Instructor-Led Extension

The slide introduces NFQUEUE and Python packet callbacks. Complete this only in the prepared lab VM because package names and Python bindings vary. The core assessed task is iptables rule evidence and PCAP verification.

- Explain that NFQUEUE passes matching packets to a userspace program.
- Identify where a callback can accept, modify or drop the packet.
- Document the rule, queue number, process and cleanup command.
- Never leave an NFQUEUE rule active without a listening process because traffic may stall.

15. Forensic Interpretation Questions

1. Why does DROP commonly cause SYN retransmissions and a timeout?
2. How would a REJECT rule differ in the packet capture and client output?
3. Why are firewall counters valuable corroborating evidence?
4. What evidence would show that the web server was down rather than firewall-blocked?
5. What is the risk of deleting a rule by line number after other rules have changed?

Required Screenshots Checklist

- ☐ Original ruleset and hash.
- ☐ Network addresses and Apache listener.
- ☐ Successful baseline curl.
- ☐ Allowed HTTP capture handshake/request/response.
- ☐ Inserted DROP rule and verification.
- ☐ Blocked curl timeout.
- ☐ Blocked capture showing retransmissions/no response.
- ☐ Rule packet/byte counters after test.
- ☐ Allowed-versus-blocked comparison.
- ☐ Rule removal and restored access.

Student Report Template

6. Executive summary.
7. Lab network and scope.
8. Original firewall state and evidence hashes.
9. Allowed baseline analysis.
10. Rule implementation and verification.
11. Blocked traffic analysis and counter correlation.
12. DROP versus REJECT discussion.
13. Cleanup and restored service.
14. Conclusions and appendix.

Submission Requirements

- One PDF report named CIP-B103-Lab6_RegNo_FullName.pdf.

- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and ruleset preservation	15	Original rules exported; narrow rule; clean restoration.
Baseline capture	15	Complete allowed HTTP session.
Firewall rule execution	20	Correct insertion, verification, counters and removal.
Blocked traffic forensics	25	Accurate packet/curl/counter correlation.
Interpretation	15	DROP, REJECT, service-down and evidence limitations.
Report quality	10	Professional screenshots and narrative.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
INPUT chain	Traffic destined for the local host.
DROP behaviour	Packet silently discarded; client often retransmits then times out.

Checkpoint	Expected Observation
REJECT behaviour	Firewall sends an error/reset, producing faster failure.
Counter evidence	Packet/byte counters increase when the rule matches.
Allowed session	SYN, SYN-ACK, ACK, HTTP request and response.
Cleanup	Delete the exact rule specification and confirm access returns.

Command Reference Summary

Command	Purpose
iptables -L -n -v --line-numbers	List rules and counters.
iptables-save	Export the current ruleset.
iptables -I INPUT 1 ... -j DROP	Insert a narrow blocking rule.
iptables -C ...	Check whether a rule exists.
iptables -D ...	Delete an exact rule.
tcp.analysis.retransmission	Filter retransmitted TCP packets.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.